

第 3 章 · LLM Serving

推理系统

Weiming Supercomputing Team
Linux Club of Peking University

3.1 推理系统的核心

原理、目标、指标、Benchmark

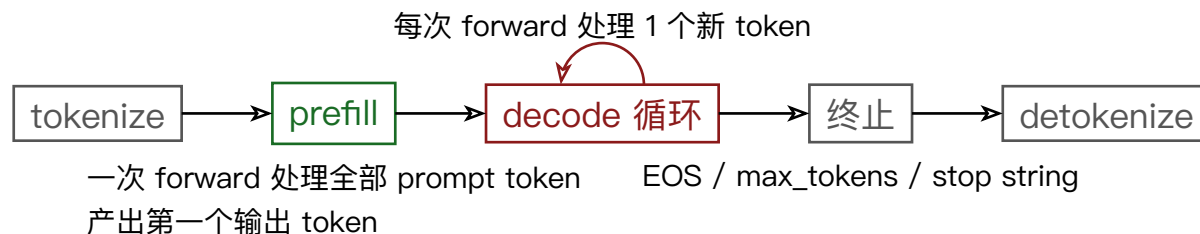
Weiming Supercomputing Team
Linux Club of Peking University



- **推理的计算形态**: 自回归生成、prefill/decode 两阶段、KV cache
- **定量分析方法**: FLOPs、访存量、arithmetic intensity 与 roofline
 - 贯穿全课的结论: prefill 是 compute-bound, decode 是 memory-bound
- **评价体系**: 指标定义与 benchmark 方法

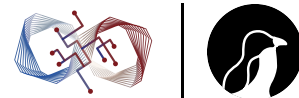
后续各节的优化均以本节的分析为基础

3.1.1 一次请求的生命周期



两个阶段的计算形态不同：

- **Prefill**: L 个 token 一起过一遍模型；线性层是矩阵乘矩阵（GEMM, $M = L$ ），attention 是 $L \times L$ 的下三角计算
- **Decode**: 每步只有 1 个新 token；线性层退化为向量乘矩阵（GEMV, $M = 1$ ），attention 是 1 个 query 对全部历史 K/V



Decode 第 t 步的 attention 需要 token $1..t$ 的所有 K 、 V

直接的说法是“缓存比每步重算便宜”，但这跳过了三个更根本的问题：

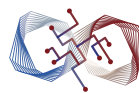
- 缓存为什么是**合法**的
- 为什么缓存的恰好是 **K 和 V**
- 丢弃之后为什么只能靠**重算**找回



- Causal mask 使第 l 层第 t 个位置的 hidden state 只依赖 token $1..t$
- 每一层都如此，归纳到整个网络：前缀所有位置的全部中间激活（包括 K、V）在该 token 被处理的那一刻就永久确定
- 之后无论追加多少新 token 都不会改变

对照没有这个性质的结构：双向 encoder 中追加一个 token 会改变所有位置的激活，任何缓存立即失效

自回归 decoder 的 KV cache 依赖因果性，是结构性质而非实现技巧



- token 之间交换信息的唯一机制是 attention
- attention 读取历史只经过两个线性投影：用 K 算权重、用 V 取内容
- 历史 token 的 hidden state、Q、MLP 激活，在它所在的那一步之后不再被任何后续计算引用
 - Q 只在本步使用；attention 对历史只读，且只读 K/V

每层的 $\{K, V\}$ 是继续生成所需的最小充分状态：缓存了它们，就缓存了历史对未来的全部影响；其余中间量都可以安全丢弃



- $K_{l,t}$ 是 $h_{l-1,t}$ 的线性像
- $h_{l-1,t}$ 需要 token t 逐层跑过前 $l-1$ 层才能得到
- 其中每一层的 attention 又要读取前缀 $1..t-1$ 在该层的 K/V
- 递归展开：重建任何一层的 K/V 等价于对整段前缀重新执行 prefill

不存在从 token id 直接映射到深层 K/V 的捷径

重算的代价 = 一次前缀 prefill —— “重算 vs 换入”取舍的本质 (3.2、3.4)



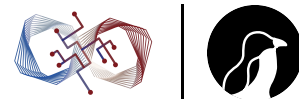
代价对比：

- 不缓存：第 t 步需要一次长度 t 的完整 forward，生成 N 个 token 总计 $O(N^2)$ 次 token-forward
- 缓存：每 token 只被完整计算一次，代价是显存中保留一份随长度线性增长的状态

KV cache 就是 transformer 的循环状态，各方案在“状态大小”与“信息保真”之间取不同的点：

方案	每 token 状态	信息保真
RNN / 线性注意力	固定大小（前缀压缩进向量）	有损
MLA (3.2)	低秩压缩的 latent	近似无损
MHA / GQA	完整（或分组共享的）K/V	无损
DSA (3.3)	状态不减，每步只读子集	按 query 选择

推理系统因此是有状态的：每个进行中的请求占用一块随长度线性增长的显存



$$\text{bytes/token} = 2 \text{ (K 和 V)} \times n_{\text{layers}} \times n_{\text{kv_heads}} \times \text{head_dim} \times \text{dtype_bytes}$$

以 Llama-3.3-70B (80 层, GQA: 8 个 KV head, head_dim 128, BF16) 为例:

- $2 \times 80 \times 8 \times 128 \times 2 \text{ B} = \mathbf{320 \text{ KB/token}}$
- 单个 128K 上下文的请求 $\approx 320 \text{ KB} \times 131072 \approx \mathbf{40 \text{ GB}}$, 接近 H100 单卡显存 (80 GB) 的一半

对照 MLA (DeepSeek V3 系):

- 每层 512 维 latent + 64 维解耦 RoPE key = 576 元素/token/层, FP8 下 576 B
- 61 层约 $\mathbf{34 \text{ KB/token}}$, 比上述 GQA 配置小约一个数量级 (机制见 3.2, kernel 见 3.7)

GQA 本身已是架构侧压缩: 8 个 KV head 对 64 个 Q head 已压缩 8 倍; MQA、GQA、MLA、DSA 属于同一方向上的四代演进

3.1.2 每 token 的 FLOPs



对参数量为 P 的 dense 模型 (MoE 取激活参数量):

$$\text{FLOPs/token} \approx 2P$$

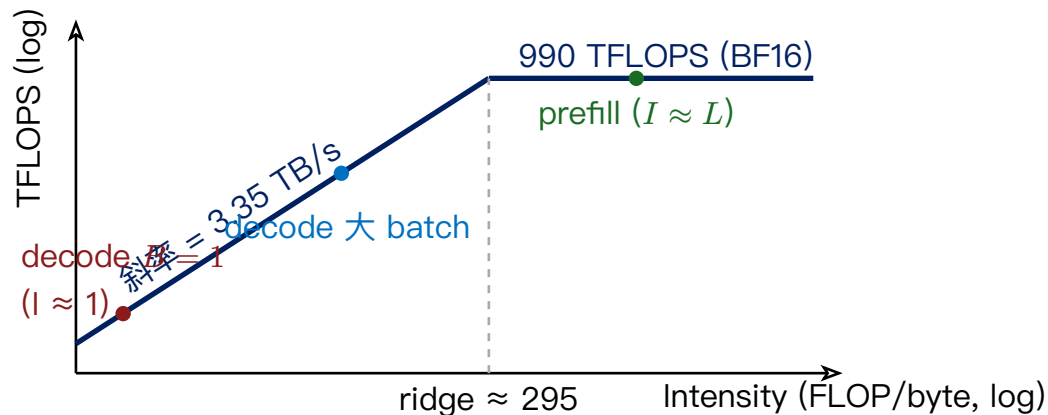
- 每个参数参与一次乘加
- attention 中与上下文长度成正比的部分: 短上下文下相对 $2P$ 是小量, 长上下文时不可忽略 (3.3), 此处按 $2P$ 记



生成 1 个 token (batch = 1), 一次 forward 从 HBM 读取:

- **全部权重**: $P \times \text{dtype_bytes}$, FP16 下 $2P$ 字节
- **该请求的全部 KV cache**: 随上下文长度线性增长
- 激活读写: batch = 1 时相对前两项可忽略

decode 每步访存量由权重与 KV 两项决定, 与计算了多少 FLOPs 无关



- Intensity = FLOPs / Bytes; decode (batch = 1, FP16) $\approx 2P / 2P = 1$ FLOP/byte
- H100 SXM: BF16 约 990 TFLOPS, HBM3 约 3.35 TB/s, ridge point ≈ 295 FLOP/byte
- decode batch=1 远低于 ridge point: 可达性能 $\approx 3.35 \text{ TB/s} \times 1 \text{ FLOP/B} \approx 3.35 \text{ TFLOPS}$, 算力利用率约 0.3%

单请求 decode 的延迟下界

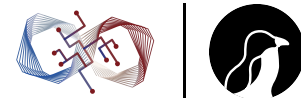


每步至少读一遍全部权重：延迟下界 = 权重字节数 / 带宽

	70B FP16	代入
权重字节数	2P	140 GB
HBM 带宽 (H100)	—	3.35 TB/s
每 token 延迟下界	2P / 带宽	$\approx 42 \text{ ms}$
单请求速度上限	—	$\approx 24 \text{ token/s}$

这个下界与 kernel 实现质量无关；在此约束下提升单请求速度的途径：

- 压缩权重（量化）
- 让一次权重读取产出多个 token（投机解码，3.8）
- 更高带宽的硬件



B 个请求同时 decode: 权重只读一次, 服务 B 个 token



B 个请求同时 decode: 权重只读一次, 服务 B 个 token

$$\text{Intensity} \approx \frac{2PB}{2P + B \cdot \text{kv_bytes}}$$



B 个请求同时 decode: 权重只读一次, 服务 B 个 token

$$\text{Intensity} \approx \frac{2PB}{2P + B \cdot \text{kv_bytes}}$$

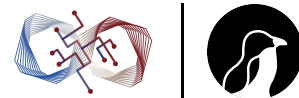
- **权重项 $2P$** : 被 B 摊薄, B 越大越接近 compute-bound



B 个请求同时 decode: 权重只读一次, 服务 B 个 token

$$\text{Intensity} \approx \frac{2PB}{2P + B \cdot \text{kv_bytes}}$$

- **权重项 $2P$** : 被 B 摊薄, B 越大越接近 compute-bound
- **KV 项 $B \cdot \text{kv_bytes}$** : 不被摊薄, 每个请求的 KV 私有, B 个请求读 B 份



B 个请求同时 decode: 权重只读一次, 服务 B 个 token

$$\text{Intensity} \approx \frac{2PB}{2P + B \cdot \text{kv_bytes}}$$

- **权重项 $2P$** : 被 B 摊薄, B 越大越接近 compute-bound
- **KV 项 $B \cdot \text{kv_bytes}$** : 不被摊薄, 每个请求的 KV 私有, B 个请求读 B 份

$B \cdot \text{kv_bytes}$ 接近权重大小时, KV 读取成为主要带宽开销



- 提高 decode 吞吐的前提是攒到足够大的 batch —— continuous batching (3.4) 的出发点
- batch 上限受显存约束 (B 个请求的 KV 必须放得下) —— 消除 KV 显存碎片 (PagedAttention, 3.2) 等价于提高吞吐
- GQA/MLA/DSA 压缩 KV 不仅节省显存, 也直接减小分母中的 kv_bytes, 提高大 batch 下的 intensity
- 长上下文场景中 KV 项占主导: 即使 batch 打满, decode 仍是 memory-bound (3.3)



- Prefill 一次处理 L 个 token: 权重读一次做 L 份计算, $\text{intensity} \approx L$ (序列内的 token 相当于 batch)
- L 超过几百即越过 ridge point, 进入 compute-bound 区

	Prefill	Decode
计算形态	GEMM ($M = L$)	GEMV/小 GEMM ($M = B$)
瓶颈	Tensor core 算力	HBM 带宽 (权重 + KV)
优化方向	高 MFU 的 GEMM/attention kernel、并行切分	增大 batch、压缩 KV、量化、投机解码、通信隐藏

两阶段瓶颈不同, 且混合 batch 效率不佳, 是 PD 分离 (3.5) 的动机



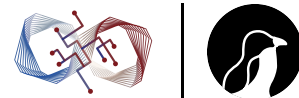
- **MFU** (Model FLOPs Utilization) = 实际有效 FLOPs / 硬件峰值 FLOPs
 - 适合衡量 prefill 与训练
- **MBU** (Model Bandwidth Utilization) = 单位时间实际搬运的权重与 KV 字节数 / 硬件峰值带宽
 - decode 阶段的 MFU 上限本身就低，衡量实现质量应看 MBU
 - 较好的 decode 实现 MBU 通常在 60—80% 以上

3.1.3 吞吐、延迟、成本



- 吞吐（每 GPU 的 output tokens/s）近似决定成本：吞吐翻倍即每 token 成本减半
- 延迟决定单请求体验
- batch size 同时作用于两者：batch 越大，intensity 越高、吞吐越高，但每步 forward 变长、排队变长，单请求延迟上升

吞吐数字必须附带延迟约束才有意义。正确的比较方式：吞吐—延迟曲线（x 轴 TPOT 或 TTFT，y 轴吞吐），或“给定 SLO 下的最大吞吐”



- **Online / interactive**: chat、coding agent、API serving
 - 有 SLO (例如 $TTFT < 2s$ 、 $TPOT < 50ms$)
 - 目标: SLO 约束下最大化吞吐
- **Offline / batch**: 评测、合成数据、批量处理、RL rollout 的部分形态
 - 无延迟约束
 - 目标: 纯吞吐与成本, 尽可能增大 batch 使 decode 接近 compute-bound

两类负载对应不同的配置与架构选择 (例如 offline 通常不需要 PD 分离); 每项技术都可以按“对哪类负载有效”归类



多轮对话与 agent 负载的结构特征：

- 每一轮请求携带完整历史（system prompt、工具定义、此前所有轮次），新增内容只是末尾一小段
- 上一轮的 KV cache 还在时，本轮 prefill 只需计算未命中的后缀部分

prefix cache 命中率是与长度分布、到达过程并列的负载核心参数：

- DeepSeek 公布的线上统计：输入 token 命中率约 56%
- agent 与 coding 类负载前缀几乎完全复用，命中率通常更高



- **Prefill 有效计算量按 $(1-h)$ 折减**: $\text{FLOPs} \approx (1-h) \times 2P \times L$
 - h 接近 1 时“prefill compute-bound”弱化，瓶颈向 decode 与 KV 的容量、搬运转移
- **TTFT 的构成改变**: 命中的 KV 必须在请求落地的机器上（或能低成本取到）
 - 高命中率负载下 TTFT 取决于缓存驻留位置与调度，而非 prefill 算力 —— cache-aware routing 与多级 KV 存储的动机 (3.2)
- **KV cache 从请求内的临时状态变为跨请求的长生命周期资产**
 - 保留多久、放在哪级存储、如何逐出 —— 3.2 “以 KV cache 为核心的系统”的含义



- Goodput = 单位时间内满足 SLO 的请求（或 token）数，超时请求不计入
- 系统可能吞吐高但 P99 TTFT 超标，此时 goodput 低于吞吐
- PD 分离相关工作与 Dynamo 的评测均以 goodput/SLO 为主要口径

3.1.4 指标定义



指标	定义	主要影响因素
TTFT	请求发出到收到第一个 token	排队时间、prefill 时长（正比于 prompt 长度）、prefix cache 命中率
TPOT	$(E2E - TTFT) / (\text{输出 token 数} - 1)$, 均值	decode batch 大小、KV 长度、是否被 prefill 抢占
ITL	相邻两个 token 之间的间隔，是一个分布	同上；chunked prefill 与投机解码会引入抖动
E2E Latency	$TTFT + \sum ITL$	—



- **TPOT 是均值, ITL 是分布**: chunked prefill 或投机解码下 TPOT 可能不变但 ITL 出现周期性尖刺; 用户感知的卡顿反映在 ITL P99 上
- **input 与 output tokens/s 分开报告**: agent 类负载 input:output 常在 10:1 以上, 合并成 total tokens/s 会使对比失真
- **生产环境看 P50 / P90 / P99 分位数**, 均值参考价值有限
- **成本口径 \$/1M output tokens**: H100 按 \$2/h、单卡 3000 output tokens/s 稳态 $\approx$ \$0.19/1M output tokens; 上下文越长、batch 越小, 单位成本越高
- **输入按命中与未命中分开计价**: 命中的输入 token 不消耗 prefill 算力, 只消耗 KV 的存储与读取; 商业 API 对 cache 命中定价约为未命中的 1/10, 价差反映真实资源消耗差异

3.1.5 负载的三个定义要素



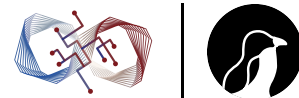
- **长度分布**：输入/输出 token 长度的分布
 - 短对话（ShareGPT 类，数百 in/out）、长文档（数万 in、数百 out）、agent/coding（数万到数十万 in、多轮、前缀高度重复）的系统行为差异很大
 - 工业界趋向使用真实 trace（如 Mooncake 开源的生产 trace）而非合成分布
- **到达过程**：
 - closed-loop（固定并发）：自带背压，不会出现队列持续增长的过载状态，会系统性高估服务能力
 - open-loop（泊松过程或真实时间戳回放）：测 SLO 需要扫描到达率，找 goodput 拐点
- **采样与终止**：greedy 还是 sampling；输出长度是否用 ignore_eos + 固定 max_tokens 控制
 - 不控制输出长度时不同引擎生成长度不同，吞吐不可比



- **vllm bench serve**、**SGLang bench_serving**: 通过 OpenAI 兼容 API 施压, 支持数据集与到达率配置
 - 应测在线 serving 全链路 (HTTP、tokenize、调度); 引擎内部的 offline benchmark 只反映 kernel 层能力
- **NVIDIA genai-perf**: 跨引擎统一口径时常用
- **录制并回放生产 trace**: 最接近真实负载的方法



- **Prefix cache 污染**: 数据集 prompt 重复或共享前缀, 且对比双方 prefix caching 开关不一致 → TTFT 与吞吐虚高
- **Warm-up 未剔除**: CUDA graph 捕获、JIT 编译 (DeepGEMM 类尤其明显)、cache 预热发生在前若干请求
- **只报吞吐不报延迟约束**: 相当于只报告吞吐-延迟曲线的最右端点
- **扫描范围不足**: 单点数字无法反映曲线形状, 需从低负载扫到过载
- **配置不对等**: 精度 (FP8 vs BF16)、并行度、max_num_batched_tokens、投机解码开关任一不同, 结果反映的是配置差异而非引擎差异
- **压测时间过短**: 碎片累积、cache 逐出、显存水位需要 30 分钟以上持续负载才能暴露



- 硬件与互联、模型与精度、并行配置
- 引擎版本与关键参数
- 数据集长度分布、到达过程与扫描范围
- TTFT / TPOT / ITL 的 P50 / P99
- input 与 output 吞吐分别报告
- 若有 SLO: goodput 曲线



$$\text{decode intensity} \approx \frac{2PB}{2P + B \cdot KV}$$

- 吞吐优化的各类手段都作用于这个比值
- Prefill compute-bound, decode memory-bound —— batching、PD 分离、kernel 设计等架构分歧的共同根源
- 吞吐必须附带延迟约束报告；生产指标是 goodput
- $B \times KV$ 项不随 batch 摊薄、随上下文线性增长：KV cache 的存储与管理是下一节（3.2）的内容

3.2 以 KV Cache 为核心的系统

存储、复用、容量、架构压缩

Weiming Supercomputing Team
Linux Club of Peking University



3.1 已建立的两个事实：

- KV cache 是随上下文线性增长的状态 (70B GQA 约 320 KB/token)
- agent 类负载下它进一步成为跨请求的长生命周期资产

本节围绕 KV cache 回答四个问题：

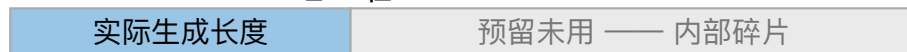
- 怎么分配与存储 —— PagedAttention (3.2.1)
- 怎么跨请求复用 —— prefix caching (3.2.2)
- 单机放不下怎么办 —— 多级存储与 offload (3.2.3)
- 如何从模型架构上减小它 —— MLA (3.2.4)

3.2.1 连续预分配的问题

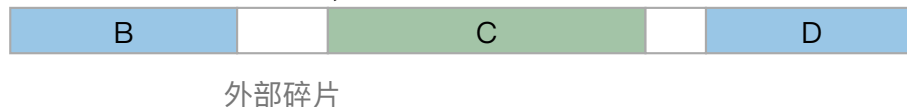


早期实现（HF transformers、FasterTransformer 时期）为每个请求按 `max_seq_len` 预分配连续显存：

请求 A（预分配 `max_seq_len`）：



显存池（请求大小不一，释放后留下空洞）：

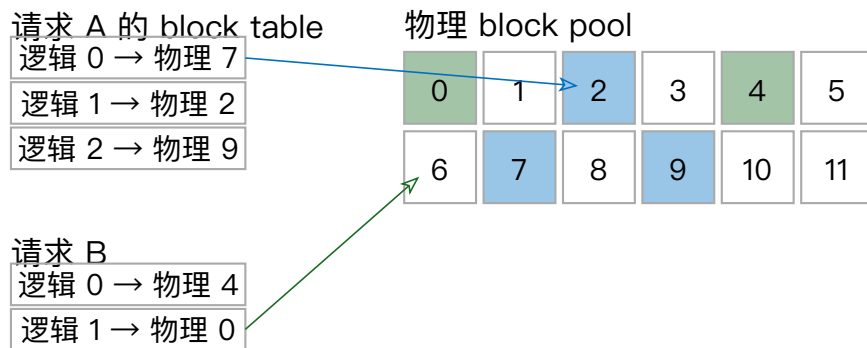


- **内部碎片**：输出长度未知，必须按最大长度预留；实际长度往往远小于预留值
- **外部碎片**：分配块大小不一，释放后留下无法复用的空洞

vLLM 论文对当时系统的测量：KV 显存有效利用率可低至 20—40%。按 3.1 的 intensity 公式，KV 显存利用率直接决定 batch 上限 B —— 碎片浪费等价于吞吐损失

操作系统虚拟内存分页的思路搬到 KV cache 管理：

- KV 按固定大小的 **block** 存储 (vLLM 默认 16 token/block)，物理上不要求连续
- 每个请求维护一张 **block table**，记录逻辑块到物理块的映射
- 分配按需进行：decode 每写满一个 block 再申请下一个；请求结束后物理块归还全局 block pool



结果：内部碎片至多 1 个 block (< 16 token)，外部碎片消除，显存利用率接近 100% —— 此后所有主流引擎的标配



Paged 布局要求 attention kernel 通过 block table 间接寻址，访存从连续变为按块 gather：

- attention kernel 必须显式支持 paged 布局（FlashAttention/FlashInfer/FlashMLA 均提供 paged 接口），block table 的读取与地址计算进入 kernel 关键路径
- **page size 的权衡：**
 - page 越小：分配粒度细、浪费少、prefix 复用粒度细
 - page 越大：block table 小、间接寻址开销低、利于访存合并与 TMA 传输

部分 kernel 对 page size 有硬性要求：

- FlashMLA 要求 64；DeepSeek V3.2 的 indexer cache kernel 要求 64，而 token 级稀疏读取算子要求 1
- SGLang 为此在同一个 attention backend 中同时支持两种 page size

page size 不是自由参数，而是 kernel 与内存管理之间的接口约定



Block table 的间接层使跨请求共享物理块成为可能：

- 同一 prompt 的 n 个并行采样、beam search 的分叉，共享 prompt 部分的物理块
- 配合引用计数与 copy-on-write：分叉后首次写入时复制

这也是 prefix caching (3.2.2) 的实现基础

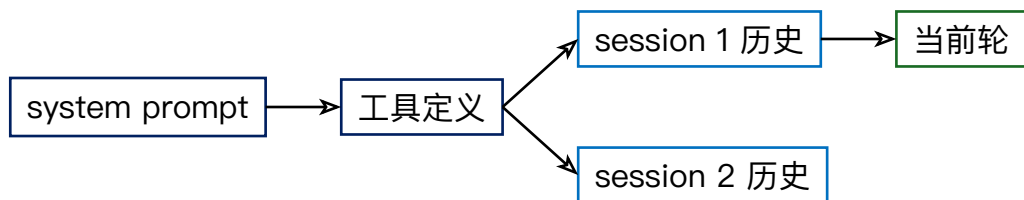
3.2.2 复用：命中的定义



- KV cache 的内容只由前缀 token 序列决定，与采样参数无关
- 两个请求的前缀 token 序列完全一致时，对应的 KV 块可以直接复用
- 命中判定是 token 级精确匹配，且按 block 粒度进行：一个 block 内 16 个 token 全部一致才能复用该块



- **Block hash 链** (vLLM): 每个物理块的 hash 由块内 token 与前一块 hash 递归计算, 前缀匹配转化为逐块 hash 查表
 - 多模态输入的图像内容、LoRA adapter id 一并纳入 hash, 避免错误命中
- **Radix tree** (SGLang 的 RadixAttention): token 序列为边、KV 块为节点组织成基数树, 前缀匹配即树上最长路径查找



树结构对“多请求共享分层前缀”的表达更直接



Cache 池满时需要逐出：

- 基本策略 LRU；radix tree 上为 leaf-first LRU（先逐出叶子，保证被逐出节点没有后代依赖）
- 正在被 in-flight 请求引用的块由引用计数保护，不可逐出

实践细节：system prompt 中的动态内容（时间戳、随机 session id）会使前缀从该位置开始全部无法命中。高命中率的前提是 prompt 模板将不变部分严格置于可变部分之前



多实例部署时 prefix cache 驻留在单个实例的显存中，命中的前提是请求被路由到持有其前缀 KV 的实例：

- 路由策略从“负载均衡”变为“命中率与负载的联合优化”
- 简单方案：session 粘滞（同一会话固定实例）
- 通用方案：路由器维护各实例的前缀索引（或近似），对每个请求计算各实例的**前缀重叠度**，在重叠度与当前负载之间加权选择
 - SGLang router 与 Dynamo 的 KV-aware routing 均属此类；Dynamo 基于 cache overlap score 与负载代价选择 worker

收益按 3.1.3 量化：命中率 h 下 prefill 有效计算量 $(1-h) \cdot 2P \cdot L$ ，TTFT 中的计算项同比例下降

3.2.3 容量：多级存储与 offload



动机：

- 命中率随可保留的 KV 总量单调上升，而 HBM 中 KV 池的容量与正在运行的 batch 直接竞争
- agent 负载下会话间隔可达分钟级：保留在 HBM 不经济，逐出后下次命中又需全量重算
- 解决方向：把 KV 下沉到更大更慢的存储层

层级	典型容量	典型带宽	相对 HBM
HBM3 (H100)	80 GB	3.35 TB/s	1
CPU DRAM (经 PCIe 5.0 x16)	TB 级	64 GB/s	2%
本地 NVMe	数 TB	7–14 GB/s	0.3%
远端存储池 (RDMA)	集群级	每链路 25–50 GB/s	1%



从下层取回 KV 是否划算，取决于传输时间与重算时间的比较：

- 传输时间 = $L \times \text{bytes/token} \div \text{层间带宽}$
- 重算时间 = prefill 该段前缀的时间 (compute-bound, 见 3.1)

两个推论：

- **KV 越小传输越划算**：MLA 的 34 KB/token 经 PCIe 传输比 GQA 的 320 KB/token 快一个数量级——架构压缩同时改善 offload 的经济性
- **传输可与计算重叠**：按层流水（第 i 层 KV 传输时计算第 $i-1$ 层）隐藏大部分传输延迟，是各传输组件的标准做法



- **Mooncake** (Moonshot / Kimi 的 serving 平台, 已开源): KVCache-centric 架构
 - 集群中 GPU 之外的 DRAM 与 SSD 组织成分布式 KV cache 池
 - prefill/decode 分离部署 (3.5), KV 经 RDMA transfer engine 在池与计算实例间流动
 - 其公开的生产 trace 是目前最常用的真实负载数据之一
- **LMCache**: vLLM 生态的 KV cache 分层组件, 支持 CPU/磁盘 offload 与跨实例共享, vLLM production stack 的一部分
- **NIXL** (NVIDIA): 点对点传输库, 统一 VRAM/DRAM/NVMe/远端之间的异步传输接口, 自动选择 NVLink、RDMA/UCX 等后端
 - 同时服务本节的 offload 与 3.5 的 PD 分离 KV 传输

3.2.4 架构侧压缩：MLA



前三小节在系统层面管理给定大小的 KV; MLA (Multi-head Latent Attention, DeepSeek V2 引入, V3/R1/V3.2 沿用) 从模型架构上减小 KV 本身:

- **KV 联合低秩压缩**: hidden state 先降维到 512 维 latent 向量 c^{KV} , K 和 V 在需要时由 c^{KV} 经上投影 W^{UK} 、 W^{UV} 恢复; 缓存的是 c^{KV} 而非 K/V
- **解耦 RoPE**: RoPE 作用在 key 上会阻碍矩阵吸收 (位置相关的旋转插在两个可合并矩阵之间), 位置信息单独走一路: 64 维、所有 head 共享 (MQA 式) 的 k^R 携带 RoPE, 与 latent 拼接



每 token 每层缓存 $512 + 64 = 576$ 个元素, FP8 下 576 字节 —— 3.1 中 34 KB/token (61 层) 的来源



推理时 MLA 有两种数学等价的展开方式，分别适配两个阶段：

	MHA 模式	MQA 模式（矩阵吸收）
做法	由 latent 恢复完整 K/V，按标准多头 attention 计算	W^{UK} 吸收进 query 投影、 W^{UV} 吸收进输出投影，attention 直接在 latent 空间进行
等价形状	标准 MHA	128 个 query head 对同一份 576 维共享“KV”做 MQA
访存	恢复引入额外 GEMM	每步 decode 只读一份 latent cache，而非 128 个 head 各一份
适用阶段	prefill (compute-bound、算力充足，attention 主体可复用成熟 dense kernel)	decode (memory-bound，带宽收益最大)



- **带宽与容量**: decode 每步 KV 读取量降至 GQA 70B 配置的约 1/10
 - 3.1 intensity 公式分母中 kv_bytes 同比例减小: 相同显存下 batch 更大, 相同带宽下吞吐更高
- **代价**: 吸收模式下等效 head_dim 576/512, 128 个 head 的 attention FLOPs 高于同规模 GQA
 - 用富余的算力换稀缺的带宽 —— 在 decode memory-bound 的前提下 (3.1.2) 是正确方向
 - “高算力密度的 MQA”是非标准形状, 标准 kernel 效率不高, 需要专用实现 (FlashMLA, 3.7)
- 工程现状: FlashMLA 支持 FP8 KV cache; latent 的量化在写入 page table 时完成



- PagedAttention 用分页加 block table 消除 KV 显存碎片，代价是 kernel 间接寻址与 page size 约束；同时提供跨请求共享物理块的机制基础
- Prefix caching 的两套索引（block hash / radix tree）加 cache-aware routing，把命中率 h 变成可工程化的量
- 多级存储把 KV 池扩展到 DRAM/NVMe/远端，取舍准则是传输时间对重算时间
- MLA 把每 token KV 压缩约一个数量级：缓存低秩 latent，decode 时以吸收模式做 MQA，以额外 FLOPs 换带宽

压缩常数项之后，KV 总量仍随上下文长度 L 线性增长，attention 计算仍需读取全部历史 —— 长上下文与 sparse attention 是下一节 (3.3) 的内容

3.3 长上下文与 Sparse Attention

成本结构、context parallel、NSA $\rightarrow$ DSA $\rightarrow$ V4

Weiming Supercomputing Team
Linux Club of Peking University



上下文长度从 4K 到 128K 再到 1M，成本结构发生质变：

- 先量化长上下文对 prefill、decode、显存三方面的影响
- prefill 侧的应对：context parallel
- 架构侧的应对：sparse attention 演进（NSA → DeepSeek V3.2 DSA → V4 hybrid attention）
- sparse attention 对推理系统各层的连锁影响

3.3.1 Prefill:attention 项超过线性层项



3.1 将每 token FLOPs 记为 $2P$ ，忽略了 attention 中与 L 成正比的部分。补上这一项：

- 每层每 token 的 attention 计算 (QK^T 与 AV 两次，各 $2 \cdot L \cdot d_{\text{model}}$ FLOPs) 合计约 $4 \cdot L \cdot d_{\text{model}}$
- 全模型 $4 \cdot L \cdot d_{\text{model}} \cdot n_{\text{layers}}$ ，与线性层项 $2P$ 的交叉点：

$$L^* = \frac{2P}{4 \cdot d_{\text{model}} \cdot n_{\text{layers}}}$$

以 Llama-3.3-70B ($P = 70\text{e}9$, $d_{\text{model}} = 8192$, 80 层) 代入： **$L^* \approx 53\text{K}$**

- 上下文超过约 5 万 token 后，prefill 的主要计算量来自 attention 而非线性层，总 prefill 时间随 L 二次增长
- 1M token 的 dense prefill 中 attention 项约为线性层项的 20 倍



3.1 的 intensity 公式中，分母的 $B \cdot kv_bytes$ 项随 L 线性增长。定量地（70B GQA，单请求 128K）：

- KV 为 40 GB，每个 decode step 读取一遍：仅此一项消耗 3.35 TB/s 带宽的 12 ms
- 40 GB/请求的占用使单卡 batch 上限降到个位数，intensity 回落到带宽受限区

长上下文 decode 的吞吐同时受两方面制约：带宽（每步读 L 份 KV）与容量（ B 被压小）

受影响项	机制	增长阶
Prefill 时间	attention FLOPs	$O(L^2)$
Decode 每步带宽	读全量 KV	$O(L)$
显存容量	KV 占用压缩 batch	$O(L)$
传输成本	offload / PD 分离搬运 KV	$O(L)$

MLA 类压缩改善的是每项中的常数，不改变增长阶；改变增长阶需要 sparse attention (3.3.3)

3.3.2 Prefill 侧: context parallel



单卡 prefill 超长序列在时间与显存上都不可行: 1M token 的激活与 KV 放不进单卡, 二次增长的计算时间远超 TTFT 约束

Context parallel (CP, 亦称 sequence parallel): 同一条序列切分到多卡, 各卡持有一段 token 的 Q/K/V

Ring attention:

- 核心问题: 每段 query 需要访问其之前所有段的 K/V
- K/V 块在参与的 GPU 之间环形传递, 每卡在计算当前块 attention 的同时接收下一块 K/V, 计算与通信重叠
- online softmax (与 FlashAttention 同源, 3.7) 保证分块结果可以增量合并

负载均衡: causal mask 下按顺序切分时, 持有序列尾部的卡要对几乎全部 K/V 计算, 头部的卡几乎无事可做

- 标准解法: 交错 (zigzag/条带) 划分, 使每卡分到的 query 位置在整个序列上均匀分布



- CP 解决的是**单请求** prefill 的时延与显存问题，属于并行策略的一种（3.6 分类中与 TP/PP 正交，可组合）
- chunked prefill（3.4）解决的是 prefill 与 decode 的**调度干扰**问题
- 二者不互相替代

长上下文 serving 的典型形态：prefill 实例内部用 CP（配合 PD 分离，3.5），decode 实例不用 CP

3.3.3 架构演进的出发点



- Attention 权重高度稀疏：每个 query 的概率质量集中在少数 key 上
- 训练后外挂的稀疏方法（按启发式丢弃 KV）精度不稳定，且丢弃后无法恢复
- 当前方向：**原生稀疏** —— 稀疏结构进入训练，模型学习在稀疏约束下工作



每个 query 通过三个并行分支访问历史，三支输出经门控合并：

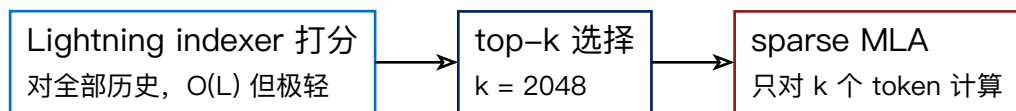
- **压缩分支**：相邻 token 块压缩成粗粒度表示，提供全局概览
- **选择分支**：按重要性选出 top 若干个原始 token 块，细粒度访问
- **滑动窗口分支**：最近若干 token 保持精确 attention

设计上按 GPU 的块访问粒度对齐（块选择而非逐 token 选择），稀疏模式对 tensor core 与访存友好

NSA 确立了“可训练、硬件对齐的稀疏”这一范式



V3.2 在 V3.1 基础上 continued training 引入 DSA —— 稀疏 attention 第一次进入 DeepSeek 旗舰模型的生产部署。“轻量打分 + 精确 top-k”两级：



- **Lightning indexer**: 每 token 一个 128 维 FP8 indexer key, 64 个 indexer head 共享同一份 key (MQA 式)
 - 每 token 仅 132 字节 (128 B key + scale), 约为 MLA latent cache (576 B) 的 1/4
 - 对每个 query 计算它与全部历史 token 的加权 ReLU 点积得分
- **复杂度**: 主 attention 从 $O(L \cdot d_{\text{latent}})$ 降为 $O(k \cdot d_{\text{latent}})$, 与 L 无关
 - indexer 打分仍是 $O(L)$, 但每 token 计算量约为主 attention 的两个数量级以下 —— 二次项系数压到接近可忽略
 - indexer 没有 value、不做完整 attention, 只输出排序依据



V4 (V4-Pro 1.6T 总参/49B 激活, V4-Flash 284B/13B) 将稀疏与压缩组合, 支撑 1M 上下文成为官方服务默认配置:

- **CSA** (Compressed Sparse Attention): 相邻 $m = 4$ 个 token 的 KV 经学习的压缩函数合并为一个 entry, 再在压缩后的 entry 序列上做 DSA 式 top-k 稀疏选择
 - 压缩减小候选集与 cache 体积, 稀疏选择保留 query 相关的细节访问
- **HCA** (Heavily Compressed Attention): 压缩率 $m' = 128$ 将历史合并为极少量 entry, 对其做稠密 attention —— 低成本的全局信息通路
- 近期 token 保留精确 attention 窗口

公开分析口径: 长上下文下 V4 计算量约为 V3.2 的 27%, KV cache 约为其 10%

- 配合 MLA (3.2), 长上下文每 token 成本从随 L 线性增长变为接近常数 —— 官方 API 长上下文大幅降价的技术基础

3.3.4 对推理系统的连锁影响（一）



DSA 是模型架构改动穿透系统所有层的完整案例：

- **KV cache 管理层**：cache 从单一 latent KV 变为两份 —— latent KV cache 与 indexer key cache, dtype、每 token 大小、访问模式都不同
 - SGLang: memory pool 中为 indexer key 与 key_scale 建独立 cache; indexer kernel 要求 page size 64、token 级稀疏读取算子要求 1, 同一 backend 并存两种 page size (3.2.1 接口约定问题的实例)
 - vLLM: KV 写入 page table 时完成 latent 与 indexer key 的量化, 为该模型维护 prefill/decode 两条独立执行路径
 - prefix caching 同步扩展: indexer cache 与 latent cache 必须一起命中、一起逐出
- **Decode 访存模式**：每步 attention 的 KV 读取从 $O(L)$ 变为固定 $k = 2048$ 个 token
 - 长上下文 decode 的带宽消耗与 L 解耦, 吞吐不再随上下文增长而下降
 - indexer 打分的 $O(L)$ 读取仍在, 但读的是 132 B/token 的小 cache



- **Offload 的可行性变化**: dense attention 下换入是全量的 (3.2.3) ;sparse 之后每步只需要 top-k 选中的 KV
 - 全量 latent cache 原则上可下沉到 DRAM 等更慢层级, 按 index 取回, GPU 只常驻小得多的 indexer cache
 - 工程难点转移为选中集合的局部性与预取: 相邻 decode step 的 top-k 重叠度决定下沉方案的实际带宽需求
 - 当前各家在长上下文 serving 上的活跃工程方向
- **Kernel 层**: DSA 的完整链路 (细节在 3.7)

DeepGEMM indexer scoring(MQA 形状、FP8、paged)→ fused top-k → FlashMLA sparse (按 indices gather)



- 长上下文同时作用于四项成本：prefill $O(L^2)$ 计算、decode $O(L)$ 带宽、 $O(L)$ 显存、 $O(L)$ 传输；常数压缩（MLA）不改变增长阶
- Context parallel 以 ring 传递加交错划分解决单请求长 prefill 的时延与显存，与 chunked prefill 解决的问题正交
- 架构演进 NSA $\rightarrow$ DSA $\rightarrow$ V4 hybrid：可训练稀疏、轻量 indexer 加精确 top-k、再叠加多档压缩，把长上下文每 token 成本压到接近常数
- Sparse attention 改写系统各层的假设：双 cache 管理、page size 并存、decode 带宽与 L 解耦、offload 从全量换入变为按 index 取回

decode 吞吐的前提始终是攒到足够大的 batch —— 请求如何组 batch 与调度是下一节（3.4）的内容

3.4 Batching 与 Scheduler

continuous batching、chunked prefill、调度机制

Weiming Supercomputing Team
Linux Club of Peking University



3.1 的结论：decode 吞吐取决于 batch 大小 B 。本节讨论如何把 B 持续维持在高位：

- static batching $\rightarrow$ continuous batching：调度粒度的变化
- prefill 与 decode 混批的干扰及 chunked prefill
- scheduler 的具体机制：队列、抢占、与 CUDA graph 及异步调度的配合

3.4.1 Static batching 及其问题



Static batching (request-level batching): 一批请求同时进入, 整批 forward 到全部完成, 再接收下一批

问题来自请求间的两种错位:

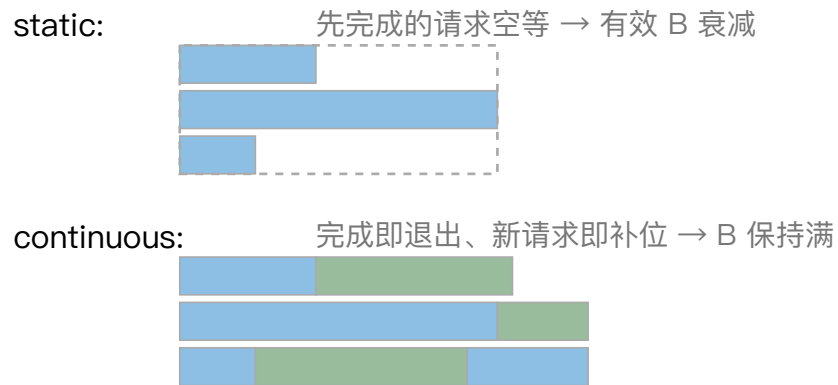
- **输出长度方差**: 先完成的请求必须等最长的请求; 等待期间仍占据 batch 槽位与 KV 显存, 有效 batch 随时间衰减
 - 输出长度分布是长尾的 (few-shot 下几个 token 到数千 token 不等), 均值意义上的浪费可达数倍
- **到达时间错位**: 新请求必须等当前整批结束才能进入, 排队延迟以“批的完成时间”为粒度

3.4.2 Continuous batching



Orca (OSDI '22) 提出 iteration-level scheduling: 调度粒度从“一批请求”降为“一次 forward step”

- 每个 step 结束后立即检查: 完成的请求当即退出并释放 KV block; waiting 队列中的请求 (KV 与 token 预算允许时) 当即加入下一个 step
- batch 成员逐 step 变化, B 持续维持在预算上限附近, 而不是随批内请求完成而衰减





批内请求长度各不相同，如何执行：

- 线性层（QKV/FFN 投影）不依赖序列结构：所有请求的 token 沿 token 维拼接成一个大 GEMM 执行
- attention 依赖各自的上下文：按请求（或按相同形状分组）分别计算
- “线性层拼接、attention 分治”是此后所有引擎的标准执行结构，也是 paged KV（3.2）按请求独立管理的前提

Continuous batching 是现代推理引擎与朴素实现之间最大的单项差距来源，vLLM、SGLang、TensorRT-LLM 的默认行为

3.4.3 混批干扰的形态



Continuous batching 使同一个 step 中同时存在两类 token:

- 新请求的 prefill token: 单请求数百到数十万个
- 在跑请求的 decode token: 每请求 1 个

一次 step 的时长由批内总计算量决定:

- 混入一个长 prefill 会使该 step 从几十毫秒拉长到数百毫秒以上
- 期间所有正在 decode 的请求停顿 → ITL 尖刺 (3.1.4 中 TPOT 均值掩盖的正是这类尖刺)

调度策略偏向 prefill 优先: decode 停顿更频繁; 偏向 decode 优先: 新请求排队、TTFT 恶化

单一资源池上这是内生冲突



将 prefill 沿序列切分为固定大小的 chunk，每个 step 按 **token 预算** (`max_num_batched_tokens`) 组装：先纳入全部 decode token（每请求 1 个），剩余预算填充 prefill chunk

效果：

- 每个 step 的计算量被预算封顶：step 时长稳定，ITL 平稳
- decode-only 的 step 带宽受限 (3.1.2)，填入 prefill chunk 恰好利用空闲算力：混合 step 的硬件利用率高于两类 step 各自单独执行
- 代价：单个 prefill 被摊到多个 step 完成，TTFT 上升

chunk 大小（token 预算）是 TTFT 与 ITL 之间的直接权衡：预算大则 TTFT 好、ITL 差，反之亦然。这个参数无法同时满足两端 —— PD 分离 (3.5) 的直接引子

3.4.4 队列与每步决策



Scheduler 维护两个集合：

- **waiting**：未开始或被抢占的请求
- **running**：持有 KV、参与 step 的请求

每个 step 前的决策：

- 继续 running 中的请求：decode 各 1 token；未完成的 prefill 取下一个 chunk
- token 预算与 KV block 余量允许时，从 waiting 提升请求（admission），分配首批 KV block
- 预算或显存不足时触发抢占



Decode 每步都要为 running 请求追加 KV block; block pool 耗尽时必须让部分请求出让显存：

	Recompute	Swap
做法	丢弃 KV，请求回 waiting，重新调度时按 prefill 重算全部前缀	KV 换出到 CPU 内存，恢复时换回
代价	重算的 FLOPs (prefill compute-bound 且吞吐高，短上下文时很短)	两次 PCIe 传输；需维护换出状态与传输通道
适用	短上下文；实现上无额外路径 (vLLM v1 采用)	长上下文更划算 (重算 $O(L^2)$ vs 传输 $O(L)$)

- 取舍准则与 3.2.3 的“换入 vs 重算”同构
- 抢占顺序一般为后到先被抢占（保护已等待更久的请求）；支持优先级的系统按优先级挑选牺牲者



多租户场景下需防止某一租户的长请求持续占据预算：

- 按租户配额
- 加权公平队列
- 对超长请求的预算上限

带 SLO 的系统还会做 admission control：预计无法满足 SLO 的请求提前拒绝或降级，避免其占用资源后超时（保护 goodput, 3.1.3）



两个 scheduler 必须感知的执行层机制（细节在 3.7）：

- **CUDA graph**：decode step 由数百个小 kernel 组成，逐个 launch 的 CPU 开销占比可观
 - 引擎按若干固定 batch 形状预捕获 CUDA graph，运行时 padding 到最近的桶后整图重放
 - scheduler 组装的 batch 形状要与捕获的桶对齐；桶的划分是 padding 浪费与显存（每桶一份 graph）的权衡
- **异步调度**：调度决策、采样、detokenize 等 CPU 工作若与 GPU forward 串行，每步之间 GPU 空转
 - 现代引擎（vLLM v1、SGLang overlap scheduler）将第 $i+1$ 步的调度与第 i 步的 GPU 执行重叠
 - CPU 侧提前准备好下一步输入元数据，GPU 步间空隙趋近于零；衡量方式即观察 GPU timeline 上 step 之间是否存在空洞



- Continuous batching 把调度粒度降到单个 forward step, 使 B 持续处于预算上限 —— 引擎吞吐的基础
- Chunked prefill 用 token 预算封顶每步计算量, 换取平稳 ITL, 并让 prefill 的算力需求与 decode 的带宽需求在同一 step 内互补; TTFT 与 ITL 的权衡由预算大小控制, 无法同时最优
- Scheduler 的核心资源是 token 预算与 KV block; 不足时以 recompute 或 swap 抢占, 取舍与 offload 同构
- 调度需与 CUDA graph 的形状桶对齐, 并与 GPU 执行异步重叠

chunked prefill 缓解了干扰, 但两阶段对硬件的需求差异在同一资源池内无法都取最优 —— 把两个阶段放到两个独立资源池, 就是下一节 (3.5) 的 PD 分离

3.5 PD 分离

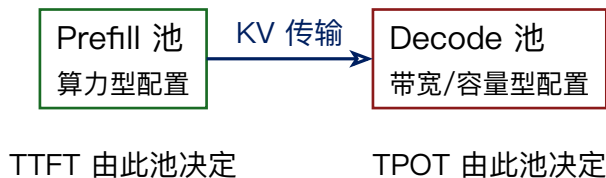
动机、KV 传输、架构与调度、适用边界

Weiming Supercomputing Team
Linux Club of Peking University



PD 分离 (prefill/decode disaggregation): prefill 与 decode 部署到两个独立的 GPU 资源池, 请求先在 prefill 池完成 prompt 处理, KV cache 传输到 decode 池后继续生成

- 概念出自 DistServe、Splitwise 等工作; Mooncake 在 Kimi 的生产环境完成工业化
- 目前是 vLLM、SGLang、TensorRT-LLM、Dynamo 的标准能力





干扰无法在单池内消除 (3.4 的结论):

- chunked prefill 把 TTFT/ITL 的冲突变成一个可调参数, 但没有消除冲突
- 同一份 token 预算内, 分给 prefill 的部分必然挤占 decode 的步频, 反之亦然
- SLO 同时约束 TTFT 与 TPOT 时, 单池只能在权衡曲线上取一点

两阶段的最优配置不同, 分离后每个池独立取最优:

	Prefill 池	Decode 池
瓶颈资源	Tensor core 算力	HBM 带宽与容量
batch 策略	无需攒 batch, 单请求即可打满算力	尽可能大的 batch
并行配置	较小 TP, 长上下文配 CP (3.3.2)	大规模 EP/TP 以扩大 KV 池与聚合带宽 (3.6)
硬件选型	算力型 GPU	带宽/显存型 GPU, 甚至不同代际



- TTFT 由 prefill 池的容量与队列决定, TPOT 由 decode 池决定
- 两个 SLO 分别监控、分别扩缩容:
 - 输入流量激增 (如批量长文档任务) → 只扩 prefill 池
 - 并发会话数上升 → 只扩 decode 池
- 单池部署中这两类压力混在一起, 只能整体扩容

3.5.2 KV 传输：量与时间量级



分离的代价：每个请求的 KV cache 必须从 prefill 池搬运到 decode 池 —— PD 分离工程上的核心问题

传输量 = $L \times \text{bytes/token}$ (3.1.1 公式)，两组对照 (400 Gbps InfiniBand $\approx$ 50 GB/s)：

	128K 上下文 KV	传输时间	结论
70B GQA	40 GB	$\approx 0.8 \text{ s}$	不可接受：dense GQA 大模型跨节点 PD 分离基本不可行，需 NVLink 域内传输或架构压缩
MLA	4.4 GB	$\approx 90 \text{ ms}$	可以接受

KV 压缩 (3.2.4) 的又一系统级收益：它同时决定了 PD 分离的可行域



实际系统中传输基本不体现在 TTFT 上，依靠两个机制：

- **逐层流水**：prefill 按层产生 KV，第 i 层计算完成即启动该层 KV 的传输，与第 $i+1$ 层的计算重叠
 - 传输在 prefill 结束后很短时间内收尾，而非结束后才开始
- **非阻塞传输**：RDMA 单边操作完成，不占用两端 GPU 的计算流
 - Dynamo 的实现：KV 经 NIXL 从 prefill 引擎的 VRAM 直接写入 decode 引擎的 VRAM，两端 forward 在传输期间继续服务其他请求



NIXL (NVIDIA Inference Transfer Library) 要点:

- 各 worker 初始化时注册全部 KV block 的内存描述符 (NIXL metadata), 经 etcd 等元数据服务分发; 此后跨节点寻址只需 block id
- 传输为 RDMA read/write 单边操作: prefill worker 直接读写 decode worker 预分配的 KV block, 无需对端 CPU/GPU 参与
- 后端自动选择: NVLink 域内走 NVLink, 跨节点走 InfiniBand/RoCE (UCX), 另有 NVMe/对象存储后端服务多级存储 (3.2.3) —— 同一套接口覆盖 PD 传输与 KV offload

布局重排:

- 两池并行配置通常不同 (例如 P 侧 TP=4、D 侧大 EP+TP), KV 在两侧的切分方式不同
- 传输时需重排 (哪些 head/层在哪张卡), 要求传输层支持非连续、多目标的散射写入 —— NIXL 强调非连续传输语义的原因

3.5.3 xPyD 配比



x 个 prefill 实例配 y 个 decode 实例，配比由负载决定：

- Prefill 池需求 $\propto$ 到达率 $\times$ 平均**未命中**输入长度（算力需求；按 3.1.3 乘以 $(1-h)$ ，prefix cache 命中率直接折减 prefill 池规模）
- Decode 池需求 $\propto$ 并发会话数 $\times$ 输出长度（KV 容量）与目标 TPOT（带宽）
- Agent/长文档类负载 input 占比高：P 池比例大；闲聊类负载相反
- 配比随流量结构漂移 $\rightarrow$ 生产系统要求 P/D 角色可动态转换（实例在两种角色间切换，而非固定划分）



Conditional disaggregation:

- 短 prefill (数百 token) 本地执行只需几十毫秒；走远程流程（入队、调度、传输、两次调度开销）反而更慢
- Dynamo：按输入长度（及当前负载）动态决定本地 prefill 还是远程 prefill，长度阈值可配
- PD 分离因此是“长 prefill 分离、短 prefill 就地”的混合形态

全局 prefill 队列与路由：

- 多个 prefill 实例之间用全局队列做负载均衡（Dynamo 基于 NATS）
- decode 侧 worker 分配好 KV block 后，把带 block id 的远程 prefill 请求推入队列；prefill worker 逐个拉取执行，完成后经 NIXL 直接写入预分配的 block
- 请求级路由叠加 3.2.2 的 cache-aware 逻辑：优先路由到前缀命中的实例



Mooncake 给出 PD 分离、prefix cache、多级存储三者的完整组合：

- 以分布式 KV cache 池为中心
- prefill 实例先从池中取回命中的前缀 KV，只计算未命中的增量部分，新产生的 KV 写回池中
- decode 实例从池/prefill 侧取全量 KV 开始生成

此时 3.2 与 3.5 的内容合并为同一套数据通路，传输层（RDMA transfer engine / NIXL）是公共底座

3.5.4 什么时候值得分离



- **值得**: interactive 负载、TTFT 与 TPOT 双 SLO、输入较长或输入输出比例波动大、有跨实例 KV 基础设施
- **不值得**:
 - offline 吞吐型负载: 没有 SLO 冲突需要解决, 分离只增加传输开销
 - 输入很短的负载: conditional disaggregation 会退化为几乎全部本地执行
 - 无高速互联的环境: 传输时间吃掉收益

二者不互斥：分离部署下 prefill 池内部仍可用 chunked prefill；decode 池内偶发的本地短 prefill 也按 chunked 混入

	Chunked prefill	PD 分离
解决方式	单池内按 token 预算折衷	双池物理隔离
TTFT/ITL	一个参数上的权衡	两侧独立优化
额外开销	无传输；prefill 拉长	KV 传输与重排；跨池调度
配置自由度	两阶段共用并行配置	两侧独立并行配置与硬件选型
部署复杂度	低	高（传输层、全局队列、配比管理）



- 公开评测多以 goodput/SLO 口径报告
- 注意组合宣传的拆分：Dynamo 1.0 宣传的 7× 吞吐提升，测量条件是 PD 分离与 wide EP (3.6) 在 GB200 NVL72 上的组合，并非分离单项的收益
- 读任何 PD 分离评测都应确认：负载的输入输出长度分布、SLO 设定、对照组是否开启 chunked prefill 并调优



- PD 分离把 chunked prefill 的单池权衡变为双池独立优化，SLO 与扩缩容随之解耦；两池的并行配置与硬件选型分别取最优
- 代价集中在 KV 传输：可行性由 $L \times \text{bytes/token}$ 与链路带宽决定（MLA 类压缩显著扩大可行域），延迟靠逐层流水与 RDMA 非阻塞传输隐藏，跨并行配置需要布局重排
- 工程形态：xPyD 动态配比（按未命中输入负载折算）、短请求条件分离、全局 prefill 队列、与 KV cache 池及 cache-aware 路由组合成统一数据通路
- 适用于双 SLO 的 interactive 负载；offline 与短输入负载不需要

两池内部各自如何用多卡承载模型 —— TP、PP、EP 的通信模式与组合 —— 是下一节（3.6）的内容

3.6 并行策略

DP、TP、PP、EP、CP 与大规模 MoE serving

Weiming Supercomputing Team
Linux Club of Peking University

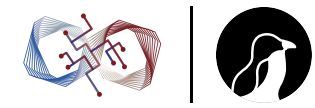


多卡的原因有二：容量（权重与 KV 放不下单卡）与性能（聚合多卡的带宽和算力）

推理并行与训练并行技术同源，但约束不同：

- 没有梯度同步，却多了 KV 状态需要随切分方式安置
- decode 每步计算量小，对通信延迟远比训练敏感

内容：按通信模式讲 DP、TP、PP、EP 四种切分，补充面向长上下文的 CP，再讨论大规模 MoE serving 的标准组合（attention DP + MoE EP）与选型逻辑



四种并行各自绑定一种通信模式，代价由拓扑决定：

原语	使用者	特点
all-reduce	TP	每层多次、延迟敏感
P2P send/recv	PP	段间单向、量小
all-to-all	EP	全池互发、拓扑要求最高
all-gather / reduce-scatter	TP 变体、CP	—

带宽层级（H100 世代）：节点内 NVLink 约 900 GB/s（双向），跨节点 RDMA 每卡约 400 Gbps $\approx$ 50 GB/s，相差约 18 倍；GB200 NVL72 将 NVLink 域扩展到 72 卡，改变的正是这条分界线的位置

基本原则：高频、延迟敏感的通信（TP 的 all-reduce）必须落在 NVLink 域内；量小或可流水隐藏的通信（PP 的 P2P）可以跨节点；all-to-all 介于两者之间，需要专门的通信实现（3.7 的 DeepEP）

两阶段的通信开销结构不同



- prefill 的通信消息大（数千 token 的激活）：代价主要在带宽
- decode 的消息极小（每请求 1 token）：代价主要在延迟

同一并行方案在两个阶段的通信开销结构不同 —— PD 分离后两池可选不同并行度的原因之一

3.6.2 TP: 切分与收益



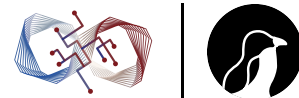
Megatron 式切法:

- MLP 的两个矩阵按 column/row 成对切分, attention 按 head 切分
- 每层只需两次 all-reduce (attention 输出投影后、MLP 第二个矩阵后), 单次通信量 = 批内 token 数 $\times d_{\text{model}} \times d_{\text{type}}$

收益:

- 权重与 KV 按 TP 度均分到各卡: decode 每步每卡的读取量变为 $1/TP$, 等效聚合带宽 $\times TP$, TPOT 理想情况下按 TP 线性下降; 同时聚合显存容量

TP 是降低单请求延迟最直接的手段



- **延迟开销**: decode 时 all-reduce 消息只有 $B \times d_{\text{model}}$, 通信是延迟主导 (单次 $10 \mu\text{s}$ 量级)
 - 每层 2 次、60–80 层累计, 是 TPOT 中与 batch 无关的固定开销, 且随 TP 度增大而增大
 - TP 度受收益递减约束, 且必须在 NVLink 域内 —— 跨节点 TP 的 all-reduce 延迟直接不可用
- **KV 复制**: attention 按 head 切分时, GQA 的 KV head 数是上限
 - Llama-70B 只有 8 个 KV head: TP=8 时每卡恰好 1 个; TP>8 则 KV head 必须复制, 均分收益消失
 - MLA 更极端: latent cache 是所有 head 共享的一份, TP 切 attention 会在每卡复制完整 latent cache —— MLA 模型的 attention 倾向 DP 而非 TP 的直接原因 (3.6.5)
- 工程: all-reduce 用引擎自带 custom kernel (小消息下 one-shot/two-shot 算法优于 NCCL ring), 并尽可能与计算重叠



切分与通信：

- 按层切成若干段，每段一组卡，段间传递 hidden states (批内 token 数 $\times$ d_model)
- P2P 单向、量小、可与计算重叠 —— PP 是**唯一对跨节点友好**的模型切分方式
- 每段只持有自己层的权重与对应层的 KV

推理中的问题：

- **Bubble**：流水线需要多个 microbatch 填充；在线 serving 负载随到达率波动，并发不足时段与段之间空转
 - 吞吐场景（持续高并发）bubble 可控；低负载或延迟敏感场景 PP 利用率差
- **TTFT 与 TPOT 的固定加项**：每个 token 顺序穿过所有段，段间通信与调度开销直接加在延迟上
 - PP 不减少每 token 的权重读取总量（与 TP 不同）：对 TPOT 没有带宽收益，只有容量收益

PP 的角色是“跨节点容量扩展”：单节点 TP 装不下的超大 dense 模型，用 TP（节点内） $\times$ PP（跨节点）承载。吞吐型 offline 负载对 PP 友好；interactive 负载在有其他选择时尽量避免深流水

3.6.4 EP:MoE 与切分



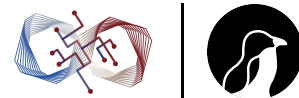
以 DeepSeek V3 系为例：每层 256 个路由专家加 1 个共享专家，每 token 激活 top-8

- EP 将专家整只分布到各卡，token 按路由结果发往专家所在卡
- 每层两次 all-to-all：dispatch (token 激活发往命中专家，FP8) 与 combine (专家输出按门控权重发回汇总，BF16)
- 通信量 = token 数 $\times$ top_k $\times$ d_model $\times$ dtype

与 TP 切 MoE 的对比：

- TP 切 MoE：每个专家切成 TP 份，但每卡仍持有**所有**专家的分片 —— decode 每步每卡读全部专家权重的 1/TP，稀疏性没有转化为带宽节省
- EP：每卡只持有 n/EP 个专家，每步只读被命中专家的权重

EP 是唯一把 MoE 稀疏性变成 decode 带宽收益的切法



DeepSeek 公开的 V3/R1 部署形态：prefill 单元 4 节点 32 卡 (EP32)，decode 单元 18 节点 144 卡 (EP144)

decode 用大得多的 EP，原因串联了前面多节的结论：

- **每卡驻留权重最小化**：EP144 下每卡只有约 2 个路由专家
 - decode 每步的权重读取量 (3.1 intensity 公式的分母) 降到最低，省出的显存全部给 KV，batch 上限随之扩大
- **专家粒度的 batch 聚合**：单卡小池子里每个专家每步只分到很少 token，GEMM 形状差
 - 144 卡的大池子把全部并发 token 汇聚后按专家分组，每个专家的 batch 足够大，专家 GEMM 从带宽受限移向算力受限 —— batching 逻辑 (3.1/3.4) 在专家维度的重演
- prefill 本身 compute-bound，不需要极端 EP，32 即可 —— 两池规模独立选择正是 PD 分离 (3.5) 提供的自由度

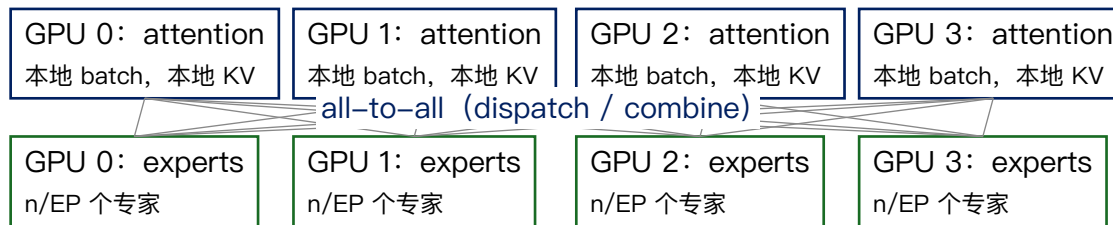


路由是数据依赖的，专家命中分布不均：热专家所在卡成为 all-to-all 与计算的瓶颈

- **EPLB** (expert parallelism load balancer)：按统计周期性重排专家放置
- 热专家复制多副本分摊流量（冗余专家），路由时在副本间均衡
- 放置策略做拓扑感知：同一 token 的多个命中专家尽量落在同节点，减少跨节点 all-to-all 流量



大 EP 部署中 attention 部分的标准做法是 **DP**：每张卡各自维护一个独立的请求 batch



- attention 完全本地计算：KV 也完全本地，无复制、无通信；只在进入 MoE 层时全池 all-to-all
- 相对给 attention 配大 TP 的优势：省去每层两次 all-reduce 的延迟；MLA 的共享 latent cache 不被复制（3.6.2 的问题消解）
- 代价：各 DP rank 的 batch 需同步推进（每步全池一起进 all-to-all），rank 间负载不均时需要 padding 或由调度器做 rank 间请求均衡 —— scheduler（3.4）在多卡场景多出的职责

标准形态：attention DP (+ 小 TP) × MoE EP，全池两次 all-to-all/层；all-to-all 的实现质量决定成立与否（3.7 的 DeepEP）

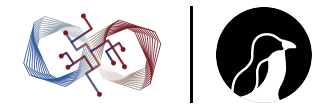
3.6.5 CP：并行策略视角



前四种并行切的是模型（权重、专家、层）或请求（DP），CP 切的是**单条序列**。3.3.2 已给出动机与 ring attention 原理，此处补三点：

- **Prefill CP 的通信代价**：N 卡下每卡每层通信量约 $(N-1) / N \times L \times \text{kv_bytes/token}$ ，随 L 线性增长；而 attention 计算量随 L 二次增长
 - 序列越长，通信越容易被计算完全隐藏 —— CP 成为少数适合跨节点的高通信并行：长序列下 P2P 环走 RDMA 也能藏进计算
- **Decode 侧 CP(DCP)**：单请求 KV 达数 GB 到数十 GB 时，单卡既放不下也读不动
 - KV cache 分片到 N 卡：query 广播，各卡对本地分片算部分 attention，结果按 online softmax 合并规则（携带 $\log\text{-sum-exp}$ 校正项）归并，通信量极小
 - 收益：单请求 KV 读取带宽聚合 N 卡，长上下文 TPOT 按 N 改善；vLLM 等已支持
 - 与 sparse attention (3.3.3) 解决同一问题的方式不同：DCP 聚合带宽读全量，DSA 减少读取量本身，两者可叠加
- **组合关系**：CP 与 TP/EP 正交；典型形态 prefill 池内 $\text{TP} \times \text{CP}$ ，MoE 层 EP 照常（CP 只作用于 attention 的序列维度）；PD 分离下 CP 主要配置在 prefill 池

3.6.6 组合与选型



各并行方式对 3.1 intensity 公式各项的作用：

	权重读取/卡	KV 读取/卡	每层通信	跨节点
TP	$\div$ TP	$\div$ TP（受 KV head 数限制）	2× all-reduce，延迟敏感	不可行
PP	不变（按段分摊容量）	按段分摊	P2P，可隐藏	友好
EP	只读命中专家	不涉及	2× all-to-all	需专门实现
DP	不变	完全本地	无（attention 部分）	—
CP	不变	$\div$ CP（单序列分片）	ring P2P / 部分结果归并	长序列下可行



- **Dense 中大模型**: 节点内 TP (≤ 8 , 对齐 KV head 数), interactive 负载到此为止; 超出单节点容量的 dense 模型加 PP 跨节点
- **MoE 大模型**: attention DP (MLA 类) 或小 TP (GQA 类) $\times$ MoE EP; PD 分离下 prefill/decode 两池独立定 EP 规模; 长上下文 prefill 叠加 CP (3.3.2)
- 决策输入: 模型结构 (dense/MoE、KV head 数、KV 大小)、负载类型与 SLO (3.1.3)、互联拓扑 (NVLink 域大小)

三者确定后, 可行的组合通常只剩一两种



- 四种并行绑定四种通信模式，可行性由“NVLink 域内 / 跨节点”的带宽与延迟分界决定；decode 的小消息使延迟而非带宽成为主要约束
- TP 均分权重与 KV 换取 TPOT 下降，受 all-reduce 延迟与 KV head 数（MLA 则是共享 latent）限制；PP 提供跨节点容量，不减少每 token 带宽需求
- EP 是唯一将 MoE 稀疏性转化为 decode 带宽收益的切法；大 EP 同时实现每卡权重最小化与专家粒度的 batch 聚合，配合 EPLB 处理热点
- CP 切分单条序列：prefill 侧通信 $O(L)$ 被计算 $O(L^2)$ 隐藏，长序列下可跨节点；decode 侧（DCP）分片 KV 聚合读取带宽，与 sparse attention 的减量方案可叠加
- Attention DP + MoE EP 是大规模 MoE serving 的标准形态，成立前提是高质量的 all-to-all 实现

并行策略确定了通信与计算的分工 —— kernel 层怎么算，是下一节（3.7）的内容

3.7 Kernel 层：Attention 与 MoE

FlashAttention 系、DeepGEMM、DeepEP、Mega MoE

Weiming Supercomputing Team
Linux Club of Peking University



前面各节确定了算什么（模型与两阶段）、在哪算（并行与调度），本节讲怎么算：

- GPU 执行模型的要点对齐
- attention kernel 主线：FlashAttention → FlashInfer → FlashMLA → DSA kernel 链路
- MoE kernel 主线：grouped GEMM → DeepGEMM → DeepEP
- 执行方式的演进：kernel launch 开销 → CUDA graph → 通信计算融合的 Mega MoE

3.7.1 GPU 执行模型要点



按“推理 kernel 高频用到的机制”做术语对齐：

- **存储层级**：寄存器 → shared memory (H100 每 SM 228 KB, 与 L1 共享) → L2 → HBM; attention/ GEMM kernel 的设计核心即数据在这个层级中的驻留策略
- **Hopper 关键机制**：TMA (异步块状 global↔shared 搬运, 地址计算卸载给硬件) ; WGMMMA (warpgroup 级异步 MMA, 128 线程为单位发射) ; thread block cluster 与 distributed shared memory
 - Blackwell (SM100)：第五代 tensor core (tcgen05)、TMEM、原生 FP4



- **Warp specialization**: block 内 warp 按角色分工 —— producer warp 专职 TMA 搬运, consumer warp 专职 MMA, 另有 warp 负责 epilogue (写回、量化、激活); 角色间以 SMEM 中的 mbarrier 同步, 形成 kernel 内流水线
 - Hopper 之后高性能 kernel 的标准结构: FlashAttention-3、DeepGEMM、Mega MoE 都采用
- **Persistent kernel**: grid 固定为 SM 数 (或其倍数), block 常驻, 内部循环从任务队列取 tile 执行; 避免多次 launch, 消除 wave quantization
- **Kernel launch 开销**: 每次 launch 的 CPU 侧代价在微秒量级, 加上驱动队列与 kernel 间隙; 单次 forward 由数百个 kernel 组成, 在 decode 中占比显著 (3.7.4 的主题)

3.7.2 FlashAttention: 核心思想



Naive attention 将 $S = QK^T$ 与 $P = \text{softmax}(S)$ (各 $L \times L$) 物化到 HBM, 读写 $O(L^2)$ 字节 —— attention 是带宽瓶颈而非算力瓶颈

两个构件:

- **Online softmax**: 分块增量计算, 维护 running max m 与归一化项 l :

$$m_{\text{new}} = \max(m, \text{rowmax}(S_{\text{block}}))$$

$$l_{\text{new}} = l \cdot \exp(m - m_{\text{new}}) + \text{rowsum}(\exp(S_{\text{block}} - m_{\text{new}}))$$

$$O_{\text{new}} = O \cdot (l/l_{\text{new}}) \cdot \exp(m - m_{\text{new}}) + \exp(S_{\text{block}} - m_{\text{new}}) \cdot V_{\text{block}} / l_{\text{new}}$$

- 已累积的输出 O 用指数因子校正后与新块贡献合并, 最终结果与整行 softmax 精确一致 (非近似)
- **Tiling**: Q 块驻留 SMEM/寄存器, K/V 块流式载入; S 与 P 只存在于片上, 从不写回 HBM
 - HBM 访存从 $O(L^2)$ 降为 $O(L \cdot d)$ (读 $Q/K/V$ 、写 O 各一遍)

版本演进: FA2 重做并行划分 (Q 沿序列切给不同 block, 长序列占满 SM); FA3 面向 Hopper 重写 —— warp specialization、TMA、WGMMMA 异步流水、GEMM 与 softmax 重叠、FP8



Decode 时 $q_len = 1$, “沿 Q 切”没有并行度可用, 改为**沿 KV 长度切** (flash-decoding / split-KV):

- KV 分成若干段, 各段由不同 block 计算部分 attention
- 最后按各段的 log-sum-exp 校正归并

这与 3.6.5 DCP 的归并规则完全相同 —— DCP 即 split-KV 的跨卡版本, 同一数学在单卡内、多卡间两个尺度上使用

定位：面向 serving 的 attention kernel 库，解决 FA 系列作为“单个 kernel”与引擎需求之间的缺口：

- paged KV 原生支持 (block table 寻址进 kernel)
- ragged batch (批内变长)
- prefill/decode/append 统一接口
- JIT 生成定制变体 (head_dim、位置编码、logits 后处理)
- plan/run 分离：host 侧先按批的长度分布规划 tile 划分与负载均衡，device 侧执行

vLLM 与 SGLang 均将其作为 attention backend 之一

MLA 吸收模式 (3.2.4) 的非标准形状: 128 个 query head 共享同一份 576 维 latent “KV”

- 标准 MHA kernel 执行会让每个 head 重复读取同一份 cache, 带宽收益丢失
- FlashMLA 的组织: 128 个 q head 视作 GEMM 的 M 维打包, latent KV 每块只从 HBM 载入一次, 供全部 head 消费
- head_dim 576 的 SMEM 压力通过分块与 warp specialization 流水消化

公开性能口径 (H800):

- decode 达 3000 GB/s 量级: 带宽受限区接近打满, MBU 接近上限
- compute-bound 形状达 660 TFLOPS
- 支持 FP8 KV cache; page size 固定 64 (3.2.1 接口约定的实例)

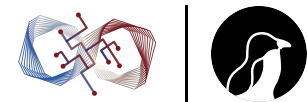


FlashMLA 接受 indices 张量 (batch, q_len, topk): kernel 按 indices 从 paged KV 中 gather 选中 token 直接进 SMEM, 不物化中间张量

DeepSeek V3.2/V4 的完整 attention 链路由三个 kernel 构成:



每步的 attention 访存从 $O(L)$ 变为 $O(k)$ (3.3.4); 代价是链路上多了两个 kernel 与 indices 的物化



- EP 下每卡每步要为本地图若干专家各算一个 GEMM，各专家分到的 token 数 M_i 不等且运行时才确定：
- 逐专家 launch 的问题：launch 开销 $\times$ 专家数；小 M_i 的 GEMM 单独执行效率低
 - Grouped GEMM：一个 kernel 内调度全部 (M_i, N, K) 子问题 —— tile 级任务列表，persistent kernel 从列表取 tile 执行，各专家的 tile 交错填满 SM

数据组织两种，对应两个阶段：

	Contiguous 布局	Masked 布局
组织	token 按专家排序后紧密排列， M_i 精确	每专家固定容量槽位，实际 token 数以 mask/count 描述，未滿部分跳过
形状	动态	静态 —— 与 CUDA graph 兼容
适用	prefill (token 多、排序开销可摊薄)	decode 路径的关键 (3.7.4) ;DeepGEMM 提供，接受期望 M 提示用于调度



DeepSeek 的 tensor core kernel 库：FP8/FP4/BF16 GEMM、grouped GEMM、indexer MQA scoring、HyperConnection 与 Mega MoE，全部 JIT 编译

- **FP8 细粒度 scaling**：按 128 通道/块粒度的 scale（而非整张量一个 scale），配合两级累加 —— tensor core 的 FP8 累加精度不足，部分和定期搬到 CUDA core 以更高精度累加
 - DeepSeek V3 系训练与推理共用的精度方案，kernel 必须原生支持才能达到可用精度
- **JIT**：运行时按具体 shape/精度特化编译并缓存，没有预编译模板集
 - 代价：首次调用的编译延迟（3.1.5 benchmark 的 warm-up 项）
 - 收益：每个 shape 都拿到特化代码，代码库小而可读 —— 与 CUTLASS 的重模板路线相反
- 实现：Hopper 用 TMA + warp specialization，Blackwell 用 tcgen05 与 FP4；公开口径 H800 FP8 至 1550 TFLOPS



EP all-to-all (3.6.4) 的专用通信库，两种模式对应两个阶段：

- **Normal(throughput)模式**：prefill 用
 - 两级转发：token 先经 NVLink 在节点内聚合/转发，再走 RDMA 跨节点，利用两层带宽的不对称
 - 目标是吞吐
- **Low-latency 模式**：decode 用
 - 取消两级聚合，纯 RDMA 点对点直发，延迟百微秒量级
 - 配 **hook 机制**：dispatch/combine 发起后立即返回，接收在后台进行，计算与通信以 micro-batch 交错重叠 —— decode step 关键路径上尽量不露出通信

两种模式都由 GPU 直接发起 RDMA (IBGDA / NVSHMEM 一系)，绕过 CPU proxy 往返；数据精度与 3.6.4 对应：dispatch FP8、combine BF16

3.7.4 Launch 开销与 CUDA graph

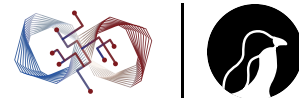


一个 decode step 包含数百个 kernel (60+ 层, 每层 attention/GEMM/归一化/量化等十余个):

- 每个 launch 的 CPU 开销加 kernel 间隙, 在小 batch decode 中可占 step 时间的两位数百分比
- CUDA graph: 一次性捕获整个 step 的 kernel 序列、参数与依赖, 之后每步单次 launch 整图重放, CPU 开销与间隙一并消除

约束是图内形状与地址必须静态:

- 引擎按若干 batch 桶分别捕获, 运行时 padding 到最近的桶 (3.4.4 中 scheduler 对齐形状桶的原因)
- KV 地址动态增长由 paged 布局吸收: graph 中固定的是 block table 指针, 表内容每步更新
- MoE 的数据依赖形状 (各专家 token 数运行时变化) 靠 masked grouped GEMM 的定容布局纳入 graph



Graph 消除了 launch 开销，但 kernel 边界本身仍在：

- dispatch 通信 kernel 完成后 GEMM 才开始，GEMM 完成后 combine 才开始
- 通信期间 SM 空闲，计算期间网络空闲；每个边界是一次全局同步
- DeepEP 的 hook 机制以 micro-batch 粒度缓解，但重叠粒度受 kernel 边界限制



DeepGEMM 中的 Mega MoE kernel 把整个 MoE 层 —— EP dispatch、Linear1、SwiGLU（含重量化）、Linear2、EP combine —— 融合为单个 kernel:

- **SymmBuffer（对称内存）**：各 rank 上地址布局一致的缓冲区，NVLink 域内以普通 load/store 直接读写远端 GPU 内存
 - 通信不再是独立的通信 kernel，而是计算 kernel 内的一条内存指令
- **Warp specialization 三角色**：dispatch warps 从对称缓冲区拉取路由到本 rank 的 token 并维护到达同步；MMA warps 执行两个 GEMM;epilogue warp groups 做 SwiGLU、重量化与 combine 写回
 - 三类角色在同一 kernel 内并发，以 tile 粒度衔接
- **Tile 粒度流水**：token 到达即参与 GEMM，GEMM 出一个 tile 即进入激活与写回 —— 没有“全部 dispatch 完成”这类全局同步点
- 精度 FP4 权重 + FP8 激活；主要面向 SM100（Blackwell），有 SM90 适配



	DeepEP（分离式）	Mega MoE（融合式）
通信形态	独立通信 kernel, GPU 发起 RDMA	kernel 内 NVLink load/store
作用域	跨节点（RDMA）+ 节点内	NVLink 对称内存域内
重叠粒度	micro-batch（hook）	tile
同步点	kernel 边界的全局同步	无全局同步，到达即算
定位	通用、与计算重叠靠 hook	端到端最优、限于域内

- 融合式的适用域由 NVLink 域大小决定：8 卡域内只覆盖小 EP; NVL72 把 72 卡纳入同一对称内存域后，decode 级别的 EP 可以整体落在融合式的作用范围内 —— 该路线当前受关注的背景
- decode 阶段 IO-bound 且 launch/同步开销占比高，是融合收益最大的场景；prefill 的大 GEMM 本身接近算力上限，融合收益有限



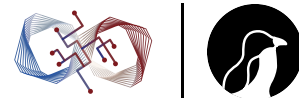
- Attention kernel 主线：online softmax + tiling 把 attention 从 $O(L^2)$ 访存降到 $O(L \cdot d)$; decode 用 split-KV, 归并规则与 DCP 同源; FlashInfer 补齐 serving 需求 (paged、ragged、JIT) ;FlashMLA 针对“128 head 共享 576 维 latent”把带宽打满, 其 sparse indices 模式与 DeepGEMM indexer、fused top-k 构成 DSA 三段链路
- MoE kernel 主线：grouped GEMM 以任务列表在单 kernel 内消化变长多专家 GEMM, masked 布局换取 CUDA graph 兼容; DeepGEMM 提供 FP8 细粒度 scaling 与 JIT 特化; DeepEP 以 normal/low-latency 两种模式分别服务 prefill 吞吐与 decode 延迟, GPU 直接发起 RDMA
- 执行方式演进：CUDA graph 消除 launch 开销 (静态形状约束由形状桶、paged 间接层、masked 布局共同满足) ;Mega MoE 进一步消除 kernel 边界, 以对称内存 + warp specialization + tile 级流水把整个 MoE 层的通信与计算融合在单 kernel 内, 适用域随 NVLink 域扩大而扩大

所有这些优化都不改变一个事实：自回归 decode 每步只产生一个 token, 单请求速度受 3.1.2 的带宽下界约束 —— 突破它需要一次 forward 验证多个 token, 即下一节 (3.8) 的投机解码

3.8 MTP 与投机解码

draft—verify 框架、无损性、draft 演进、DFlash

Weiming Supercomputing Team
Linux Club of Peking University



3.1.2 给出的单请求速度下界：每生成一个 token 至少把权重读一遍

投机解码是目前唯一绕过该下界的通用方法：让一次权重读取验证多个 token

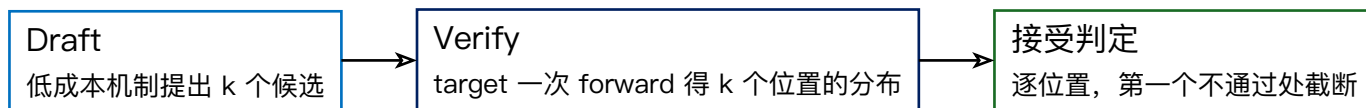
- 通用框架与无损性证明
- draft 的来源演进：独立小模型 $\rightarrow$ Medusa/EAGLE $\rightarrow$ MTP
- 并行 drafting：DFlash
- 与系统各层的交互

3.8.1 为什么 verify 几乎免费



Target 模型对 k 个 token 的一次 forward:

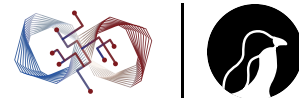
- 权重读取量与对 1 个 token 的 forward 相同 (读一遍), 计算量增加 k 倍
- decode 处于带宽受限区、算力利用率不足 1% (3.1.2): k 倍计算在算力上限之内
- “并行验证 k 个候选”的时间 $\approx$ “生成 1 个 token”的时间





投机解码是精确算法，输出分布与 target 单独解码严格一致：

- **贪心解码**：逐位置比对 draft token 与 target argmax，第一个不一致的位置直接改用 target 的 token
 - 每轮至少产出 1 个有效 token（全拒绝时也有 verify forward 给出的那一个），最多 $k+1$ 个
- **随机采样**：位置 i 的候选 x （draft 分布 q ，target 分布 p ）以概率 $\min(1, p(x)/q(x))$ 接受
 - 拒绝时从残差分布 $\text{norm}(\max(0, p-q))$ 重采样并截断
 - “接受 + 残差重采样”的混合分布恰好等于 p —— rejection sampling 的标准构造，逐位置归纳即得整条序列分布不变



设每轮平均产出 τ 个 token（平均接受长度 + 1）。有效收益是把“每 token 读一遍权重”摊薄为“每 τ 个 token 读一遍”：

$$\text{speedup} \approx \frac{\tau}{1 + c_{\text{draft}}}$$

c_{draft} = 每轮 draft 耗时 / target 单步耗时

两个改进方向由此确定：

- 提高 τ （draft 质量）—— 3.8.2
- 降低 c_{draft} （draft 效率）—— 3.8.3

3.8.2 Draft 的来源：演进



方案	做法	问题 / 特点
独立小模型	同族小模型起草（如 1B 给 70B）	分布对齐差 τ 低；需单独部署；小模型自回归 k 步耗时（ c_{draft} 大）；生产中已基本淘汰
Medusa	target 最后一层 hidden state 上外接多个轻量 head，第 i 个 head 直接预测第 i 个未来 token	无需独立模型、draft 成本极低；但各 head 独立预测、不建模候选间依赖， τ 有限
EAGLE 系	draft 在 target 的 特征 （最后一层 hidden state)上自回归：单层轻量 transformer 接收特征与已生成 token 的 embedding，预测下一位置特征，经 target 的 LM head 得到分布	特征包含比 token 多得多的信息，对齐质量显著高；EAGLE-2 置信度动态草稿树，EAGLE-3 多层特征融合；vLLM/SGLang 生产使用最广、各类新方法的对比基线
MTP	训练目标与推理 draft 双重身份（下页）	与主模型联合训练，模型自带



DeepSeek V3 的 MTP (Multi-Token Prediction)：

- 训练时为主干附加预测模块：以当前特征与后续 token embedding 为输入、经一层 transformer 预测第 2 个未来 token —— 训练信号更密（每个位置监督两个目标）
- 该模块结构与 EAGLE 的 draft 头同构，推理时直接用作投机 draft
- **与主模型同数据同分布联合训练，无需任何额外训练**
- V3 技术报告口径：第二 token 接受率约 85—90%

工程含义：MTP 权重随模型 checkpoint 一起发布，引擎加载后即可开启投机 —— 投机解码从“部署方的可选优化”变成“模型自带的功能”。DeepSeek 生产部署默认开启 MTP



- 每个位置保留多个候选可构成草稿树
- 树形 attention mask (每个节点只看到其祖先) 一次 forward 验证整棵树, 接受其中最长的通过路径
- 树加宽提高 τ , 但 verify 的 token 数按树大小增长: batch 越大、decode 越接近算力上限, 可容纳的树越小
- 生产配置随负载在“链式 (每位置 1 候选) 到小树”之间选择

3.8.3 DFlash: 并行 drafting



动机：上述方案的 draft 都是自回归的 —— k 个候选需要 k 次 draft forward，串行性使 c_{draft} 随 k 增长，限制 τ 的上探空间

机制：轻量 **block diffusion** 模型做 draft

- 以 target 的多层 hidden feature 作为条件（注入 draft 模型的 KV）
- 对“1 个 anchor token + 若干 mask token”的块做单次 forward，一次并行填出整块（8–16 个）候选 token
- draft 输出经 target 的 LM head 投影为词表分布
- 扩散模型的并行生成能力限定在 drafting 环节：draft 质量的不足由 verify 兜底，无损性不受影响；条件于 target 特征保证较高接受率



- 论文口径 (ICML 2026): 多模型多任务无损加速 6x 以上, 相对 EAGLE-3 至 2.5x
- NVIDIA Blackwell 评测口径: gpt-oss-120b 相同交互度下吞吐提升至 15x (对比 EAGLE-3)
- 已集成 SGLang、vLLM、TensorRT-LLM; 官方发布覆盖主流开源模型系的 draft checkpoint; UCSD 团队在 vLLM TPU 生态完成 TPU 实现
- 方向意义: draft 侧串行性消除后, 每轮块长度可以做大, τ 的上限被打开

3.8.4 与 batch 的关系



投机解码的本质是用算力换带宽 (verify 计算 $\times \tau$)，收益前提是 decode 有富余算力：

- 小 batch / latency 敏感场景收益最大 —— 正是 3.1.2 中离带宽下界最近的场景
- batch 增大、decode 逼近算力上限时，verify 的额外计算挤占真实吞吐，收益递减以至为负
- 生产系统据此做动态控制：按当前 batch 大小与实时接受率调整 k 或直接关闭投机
- 评测报告接受长度 τ 必须带任务分布：代码、数学类可预测性高、 τ 大；开放对话 τ 明显更低 —— 单一 τ 数字不可外推



- **Scheduler 与 KV**: 每请求每步产出 token 数从固定 1 变为 $1..k+1$ 的随机量
 - KV block 按 $k+1$ 预留; 拒绝位置的 KV 需回滚 (paged 布局下释放对应位置)
 - token 预算核算、流式输出推送节奏都要适配变长产出
 - ITL 分布呈多峰 (一次接受多个 token 后集中送出) —— 呼应 3.1.4 TPOT 均值与 ITL 分布的区别
- **CUDA graph**: draft 与 verify 是两种形状的 forward, 各自需要 graph 桶; k 可变时形状组合增多
- **大 EP 部署(3.6.4)**: verify 使每步进入 MoE 的 token 数 $\times(k+1)$, all-to-all 流量与专家 batch 同比变化
 - DeepSeek 生产选择 MTP 单步 ($k=1$): 接受率、verify 开销与通信增量之间的折中



- 投机解码利用 decode 的算力冗余，把每 token 一次的权重读取摊薄为每 τ token 一次；rejection sampling 构造保证输出分布与 target 严格一致
- Draft 演进主线是与 target 的对齐度和 draft 成本：独立小模型 $\rightarrow$ 并行 head (Medusa) $\rightarrow$ 特征级自回归 (EAGLE 系) $\rightarrow$ 训练时联合学习的 MTP (模型自带 draft)
- DFlash 用 target 特征为条件的 block diffusion 一次并行出整块 draft，消除 draft 侧串行性，是当前无损加速的最优口径
- 收益与 batch 负相关，需按负载动态启停；对 scheduler、KV 回滚、CUDA graph、EP 通信量均有连锁影响

至此各技术层已讲完 —— 最后一节 (3.9) 把它们放回具体框架中对照，梳理演进脉络，并简述 RL rollout 这一新负载

3.9 框架全景、演进与新负载

vLLM / SGLang / TensorRT-LLM / Dynamo, 时间线, RL rollout

Weiming Supercomputing Team
Linux Club of Peking University



- 把机制放回具体框架：vLLM、SGLang、TensorRT-LLM、Dynamo 在每一层的选择
- 按时间线梳理推理系统的演进主线
- RL rollout：正在反向影响引擎设计的新负载



- **vLLM**: 以 PagedAttention 起家的通用引擎；v1 架构重写后为多进程 EngineCore + 异步调度
 - 特征是广度：硬件后端最多（NVIDIA/AMD/TPU/Trainium 等）、新模型 day-0 支持惯例、attention backend 可插拔
- **SGLang**: 以 RadixAttention 起家
 - 特征是调度与前沿模型支持的深度：zero-overhead/overlap scheduler、结构化输出
 - 对 DeepSeek 系支持最深：大 EP + PD 分离复现官方部署形态、V3.2 稀疏 attention day-0 支持（3.3.4 的双 page size 工作即出自此）
- **TensorRT-LLM**: NVIDIA 官方引擎
 - 特征是 kernel 与编译深度：新硬件（Blackwell）首发优化通常最先出现，与 NVIDIA 软件栈（NIXL、Dynamo、Triton）闭环
- **Dynamo**: 不是引擎，是引擎之上的**编排层**
 - PD 分离全局调度（prefill 队列、conditional disaggregation）、KV-aware 路由、KVBM（KV block 管理与多级存储）、NIXL 传输
 - worker 可以是 vLLM、SGLang 或 TensorRT-LLM；把“单引擎之外”的部分产品化（3.2.3/3.5）



层	vLLM	SGLang	TensorRT-LLM	Dynamo（编排）
KV 管理	PagedAttention（出 处）	Paged + RadixAttention（出 处）	Paged	KVBM、多级存储、 cache-aware 路由
Prefix 复用	block hash 链	radix tree	支持	路由层按 overlap score 分配
调度	continuous batching、chunked prefill、v1 异步调度	overlap scheduler	in-flight batching	全局 prefill 队列、 xPyD
并行	TP/PP/EP/DP/ DCP	TP/PP/大 EP（复现 DeepSeek 部署形 态）	TP/PP/EP	跨实例编排



层	vLLM	SGLang	TensorRT-LLM	Dynamo（编排）
Kernel	attention backend 可插拔	FlashInfer/ FlashMLA、双 page size	自研 kernel, Blackwell 首发	—（NIXL 传输层）
PD 分离	支持	支持	支持	全局调度、 conditional disaggregation

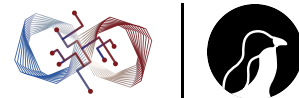
两点观察：

- 单引擎层面四家的机制清单已高度趋同：本课讲的每项技术在三个引擎中都有实现，差异在成熟度、默认值与生态
- 竞争重心正从引擎内（kernel、调度）向引擎外（多实例编排、KV 池、路由）移动 —— Dynamo、Mooncake 这类系统的出现即是标志

3.9.2 演进时间线



时间	事件	引入的能力
2022	FasterTransformer 等	手工 fused kernel, static batching
2022	Orca	continuous batching（调度粒度降到 step）
2023	vLLM	PagedAttention（KV 显存精细管理）
2023—24	SGLang;Sarathi 一系	RadixAttention（前缀复用）;chunked prefill
2024	Mooncake	PD 分离工业化, KVCache-centric 多级存储
2025	DeepSeek 开源周	FlashMLA/DeepEP/DeepGEMM, 大 EP 部署形态公开
2025	Dynamo	引擎之上的编排层（全局调度、KV 路由、NIXL）
2025	V3.2 DSA;DFlash	生产级 sparse attention; 并行 drafting



- **调度粒度细化**：请求级 → step 级 (Orca) → token 预算级 (chunked prefill)
- **显存资源化**：KV 从执行细节变为被分页、复用、分级、路由的核心资源 (vLLM → SGLang → Mooncake)
- **部署解耦**：单实例 → PD 分离 → 多池编排与独立扩缩容 (Mooncake → Dynamo)
- **模型与系统协同设计**：MLA/DSA/MTP 在模型侧为系统目标 (KV 大小、attention 复杂度、投机能力) 做架构决策，kernel 与 cache 管理随之联动 (DeepSeek 系为代表)

第四条是当前区分度最大的方向：系统优化的上限越来越多地在训练模型时就被决定

3.9.3 新负载：RL rollout



RL 后训练中样本生成 (rollout) 就是推理：主流训练框架 (verl 等) 内嵌 vLLM/SGLang 作为 rollout 引擎

构成 3.1.3 两类负载之外的第三类：无 SLO、纯吞吐，但有 serving 没有的约束：

- **权重同步**：每个训练步后新权重推送给 rollout 引擎，频率高（分钟级甚至更短）、要求快（秒级完成，否则 GPU 空等）
 - 两种形态：训练与 rollout 共卡 (colocate, 显存内切换/共享权重) 与分离部署 (NCCL/RDMA 广播权重)
 - 要求推理引擎具备高效的权重热更新路径 —— 传统 serving 没有的接口
- **Partial rollout**：长轨迹 (agentic 任务可达数十万 token) 横跨多个权重版本
 - 为控制 off-policy 程度并避免长尾轨迹拖慢整步：轨迹中断、KV 保留、换权重后续跑 —— 对 scheduler 与 KV 生命周期管理的新要求



- **Agentic rollout 的负载形态**：多轮工具调用、环境交互穿插生成
 - 前缀高度重复（prefix cache 命中率高，3.1.3 的分析适用）；并发随环境返回剧烈波动，对调度弹性要求高
- **数值一致性**：rollout 用推理 kernel、训练用训练 kernel，同一序列的 logprob 存在数值差异
 - 影响 on-policy 假设与重要性采样校正的准确性 —— “推理 kernel 与训练 kernel 的数值对齐”成为算法正确性问题

rollout 需求正反向进入引擎主干：权重热更新接口、可中断恢复的请求生命周期、与训练框架的共存部署模式，已是 vLLM/SGLang 的正式特性面



$$\text{decode intensity} \approx \frac{2PB}{2P + B \cdot KV}$$

- 把 B 维持在上限 → continuous batching 与 scheduler (3.4)
- 扩大 B 的可行域 → PagedAttention、多级存储 (3.2)
- 压缩 KV 项 → MLA (3.2)、DSA (3.3)
- 分摊分母到多卡 → TP/EP (3.6), 大 EP 把 MoE 权重项压到最小
- 逼近硬件带宽峰值、消除 launch 与同步损耗 → kernel 层 (3.7)
- 让 decode 优化不被 prefill 干扰 → PD 分离 (3.5)
- 公式之外, 用算力冗余摊薄权重读取 → 投机解码 (3.8)

模型架构 (MLA → DSA → hybrid attention → MTP) 与系统的协同, 是当前所有前沿部署形态的共同特征